UT CHETUMAL

Desarrollo y Gestión de Software

Extracción de conocimiento en bases de datos

Dr. Martín Antonio Santos Romero

T.S.U. Miguel Ángel Peña Pech

9no cuatrimestre

Práctica Autónoma 4

Proyecto de ciencia de datos 2

Índice

Introducción 3

Tareas a realizar por el alumno 4

Conclusión 9

Introducción

En el ámbito del análisis de datos y el aprendizaje automático, la capacidad de predecir variables categóricas a partir de características demográficas y socioeconómicas es fundamental para la toma de decisiones informadas. En esta actividad, se desarrolló un modelo de clasificación basado en el algoritmo de Máquinas de Soporte Vectorial (SVM), con el objetivo de predecir si los ingresos de un individuo superan o no los 50K anuales. El proceso incluyó etapas clave como la carga y exploración de datos, preprocesamiento, entrenamiento del modelo y evaluación de su rendimiento.

Tareas a realizar por el alumno

1: Analiza el siguiente caso: ¿Dormir bien mejora tu rendimiento cognitivo? Un análisis con regresión lineal.

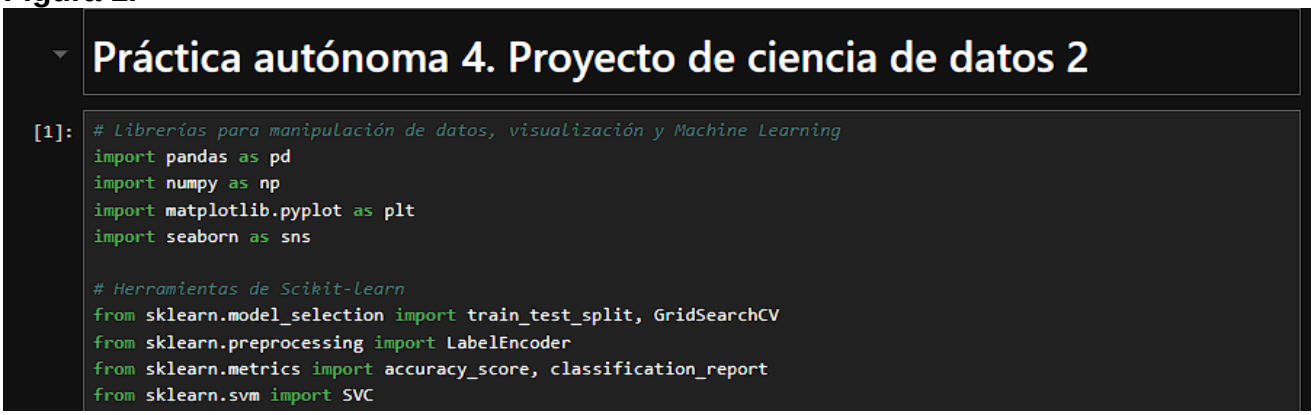
En la actualidad, el ritmo acelerado de vida, las jornadas laborales extensas y el uso constante de dispositivos digitales han provocado que millones de personas experimenten trastornos del sueño. Diversos estudios en el campo de la neurociencia cognitiva han demostrado que ***la calidad y cantidad del sueño tienen un impacto directo en el rendimiento mental***, afectando funciones como la atención, la memoria y la velocidad de procesamiento de información.

Un grupo de investigadores en neurociencia desea estudiar esta relación de forma empírica mediante un experimento controlado. Para ello, se ha recolectado un conjunto de datos de participantes que realizaron una serie de pruebas cognitivas después de haber registrado sus hábitos de sueño, actividad física, consumo de cafeína y niveles de estrés. Una de las pruebas más utilizadas en este tipo de estudios es la medición del ***tiempo de reacción*** (Reaction Time), una variable que representa cuántos milisegundos tarda una persona en responder a un estímulo (por ejemplo, presionar una tecla al ver una señal visual en pantalla).

El ***tiempo de reacción*** es una medida objetiva y estandarizada del rendimiento cognitivo, especialmente útil para detectar los efectos de la fatiga, la falta de sueño o el estrés sobre el cerebro.

Antes de comenzar, importamos las librerías necesarias. Estas herramientas nos permitirán manipular datos, entrenar modelos y evaluar resultados.

Figura 1.



Fuente: Angel Peña (2025).

train_test_split: Divide los datos en conjuntos de entrenamiento y prueba.
GridSearchCV: Ayuda a encontrar los mejores parámetros para el modelo.
LabelEncoder: Convierte variables categóricas (como "Hombre" o "Mujer") en números.
accuracy_score y classification_report: Evalúan qué tan preciso es el modelo.
SVC: Implementa el algoritmo de Máquinas de Soporte Vectorial (SVM).

El primer paso es cargar el dataset y revisar su estructura para entender qué información contiene. Elimina espacios en los nombres de las columnas y muestra las primeras filas del dataframe para inspección.

Figura 2.

```
[2]: # Ruta del archivo de Excel con los datos de entrenamiento
     ruta = r"C:\Users\king1\tareasPython\Practica autonoma 4\Ingresos_por_persona.xlsx"

     # Cargar el archivo Excel en un DataFrame
     df = pd.read_excel(ruta)

     # Eliminar espacios en los nombres de las columnas (si los hay)
     df.columns = df.columns.str.strip()

     # Mostrar las primeras filas del DataFrame para inspección
     print("Primeras filas del dataset original:")
     print(df.head())

Primeras filas del dataset original:
   edad  nivel_educ    raza     sexo  ganancias_capital  perdidas_capital  \
0    39          13   White     Male               2174                 0
1    50          13   White     Male                  0                 0
2    38           9   White     Male                  0                 0
3    53           7   Black     Male                  0                 0
4    28          13   Black   Female                  0                 0

   Horas_semana Ingresos
0            40    <=50K
1            13    <=50K
2            40    <=50K
3            40    <=50K
4            40    <=50K
```

Fuente: Angel Peña (2025).

pd.read_excel() carga el archivo Excel en un DataFrame (tabla de Pandas).
df.columns.str.strip() limpia los nombres de las columnas por si hay espacios innecesarios.
df.head() muestra las primeras filas para una revisión rápida.

Los modelos de Machine Learning requieren que los datos estén en formato numérico. Convertimos las variables categóricas (texto) a números.

Figura 3.

```
[3]: # Crear un codificador de etiquetas para convertir texto a números
     le = LabelEncoder()

     # Recorrer cada columna y transformar solo si es tipo 'object' (categórica)
     for col in df.columns:
         if df[col].dtype == 'object':
             df[col] = le.fit_transform(df[col])

     # Mostrar nuevamente las primeras filas ya transformadas
     print("Dataset con variables numéricas:")
     print(df.head())

Dataset con variables numéricas:
   edad  nivel_educ  raza  sexo  ganancias_capital  perdidas_capital  \
0    39          13     4     1               2174                 0
1    50          13     4     1                  0                 0
2    38           9     4     1                  0                 0
3    53           7     2     1                  0                 0
4    28          13     2     0                  0                 0

   Horas_semana  Ingresos
0            40         0
1            13         0
2            40         0
3            40         0
4            40         0
```

Fuente: Angel Peña (2025).

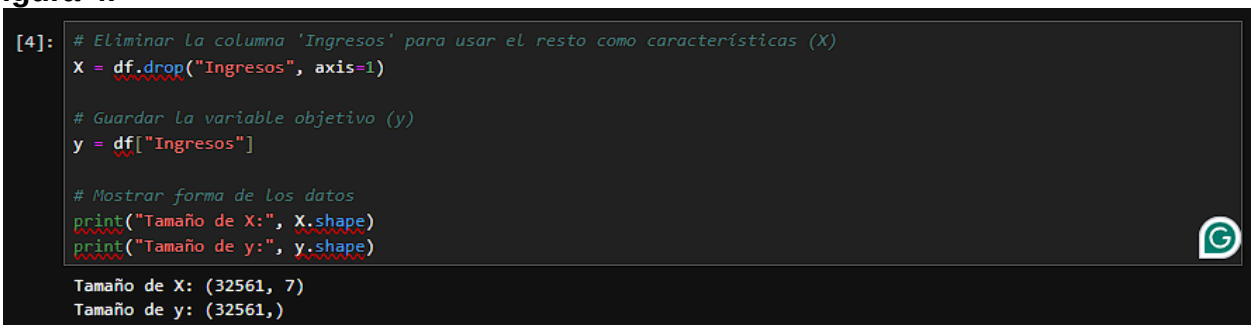
LabelEncoder() asigna un número único a cada categoría (ejemplo: "Male" → 1, "Female" → 0).
El bucle **for** recorre todas las columnas y aplica la transformación solo a las columnas de texto.

Dividimos los datos en dos partes:

Variables predictoras (X): Características como edad, educación, horas trabajadas, etc.

Variable objetivo (y): La columna "Ingresos" (≤50K o >50K).

Figura 4.



Fuente: Angel Peña (2025).

X contiene las características que el modelo usará para predecir.

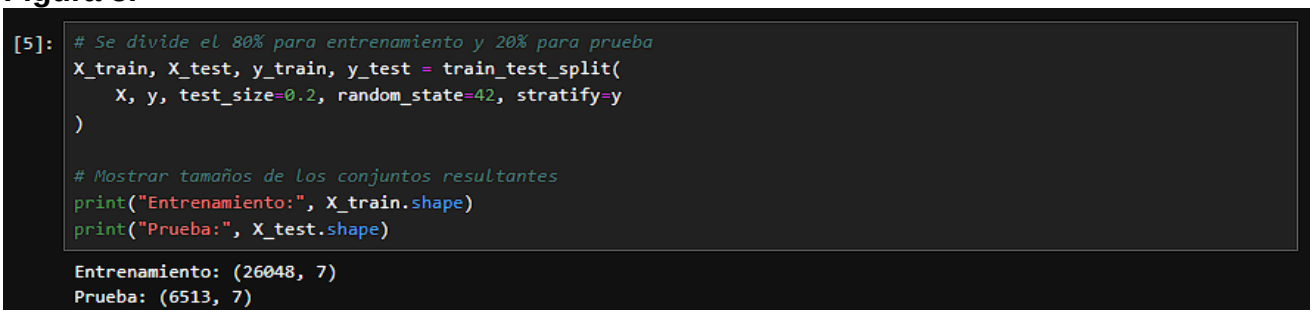
y es lo que queremos predecir (en este caso, la categoría de ingresos).

Para evaluar correctamente el modelo, separamos los datos en:

80% para entrenamiento (para que el modelo aprenda).

20% para prueba (para evaluar su precisión).

Figura 5.



Fuente: Angel Peña (2025).

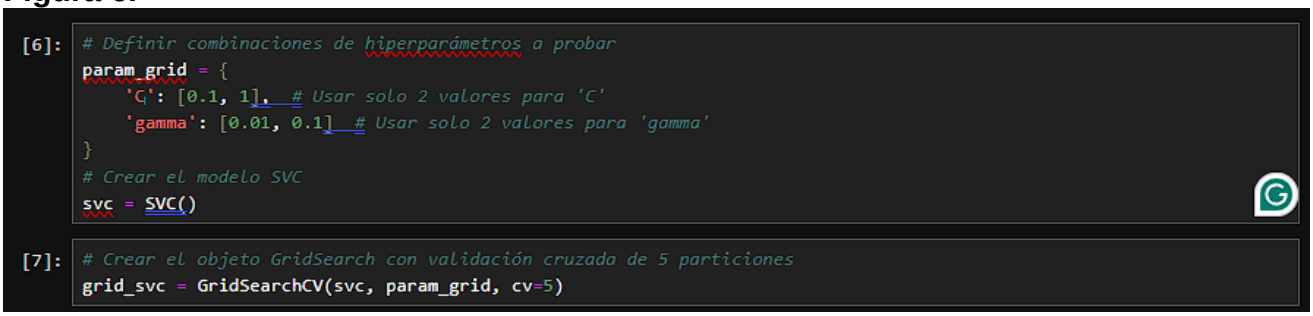
test_size=0.2 reserva el 20% para prueba.

random_state=42 asegura que la división sea reproducible.

stratify=y mantiene la proporción de clases (≤50K y >50K) en ambos conjuntos.

Usamos **Support Vector Machine (SVM)** con búsqueda de hiperparámetros para optimizar el modelo. Creamos el modelo SCV y creamos el objeto GridSearch con validación cruzada de 5 particiones.

Figura 6.



```
[6]: # Definir combinaciones de hiperparámetros a probar
     param_grid = {
         'C': [0.1, 1],  # Usar solo 2 valores para 'C'
         'gamma': [0.01, 0.1]  # Usar solo 2 valores para 'gamma'
     }
     # Crear el modelo SVC
     svc = SVC()

[7]: # Crear el objeto GridSearch con validación cruzada de 5 particiones
     grid_svc = GridSearchCV(svc, param_grid, cv=5)
```

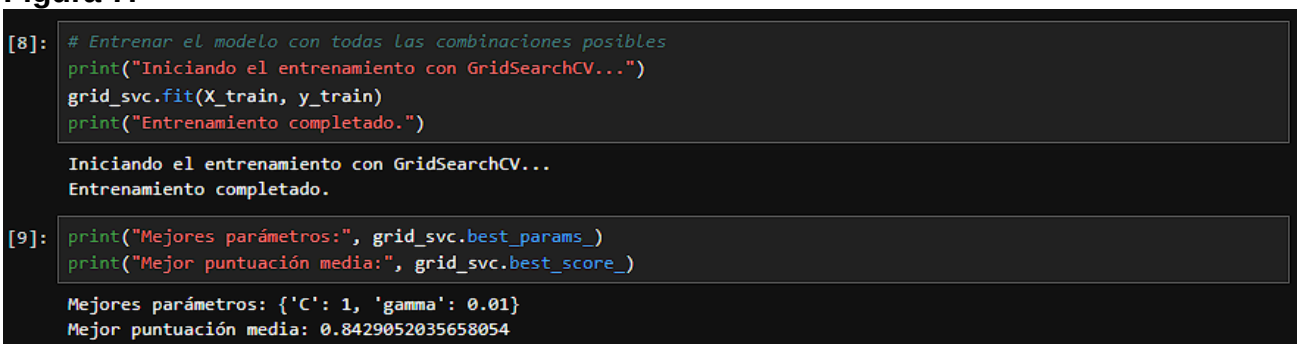
Fuente: Angel Peña (2025).

GridSearchCV prueba todas las combinaciones de **C** y **gamma** para encontrar las mejores configuraciones.

cv=5 usa validación cruzada de 5 particiones para evitar sobreajuste.

Después del entrenamiento, evaluamos su rendimiento con los datos de prueba y visualizamos los resultados.

Figura 7.



```
[8]: # Entrenar el modelo con todas las combinaciones posibles
     print("Iniciando el entrenamiento con GridSearchCV...")
     grid_svc.fit(X_train, y_train)
     print("Entrenamiento completado.")

     Iniciando el entrenamiento con GridSearchCV...
     Entrenamiento completado.

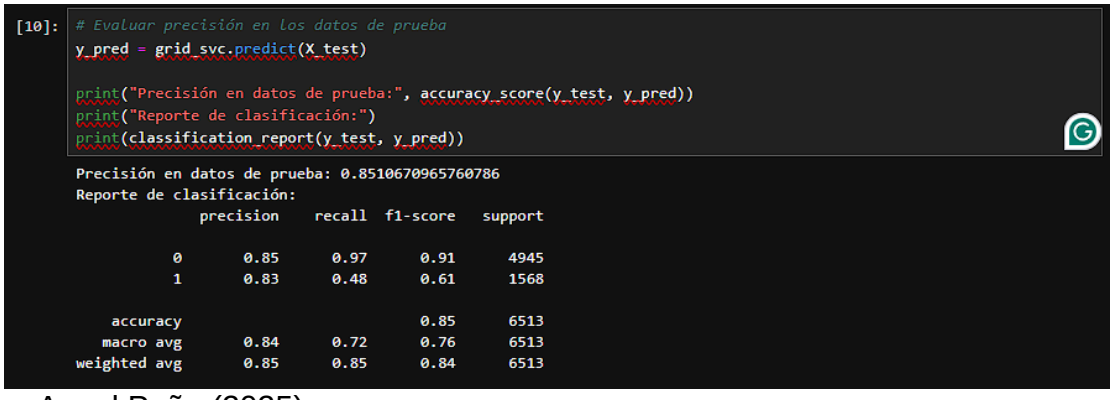
[9]: print("Mejores parámetros:", grid_svc.best_params_)
     print("Mejor puntuación media:", grid_svc.best_score_)

     Mejores parámetros: {'C': 1, 'gamma': 0.01}
     Mejor puntuación media: 0.8429052035658054
```

Fuente: Angel Peña (2025).

Evaluamos la precisión en los datos de prueba.

Figura 8.



```
[10]: # Evaluar precisión en los datos de prueba
      y_pred = grid_svc.predict(X_test)

      print("Precisión en datos de prueba:", accuracy_score(y_test, y_pred))
      print("Reporte de clasificación:")
      print(classification_report(y_test, y_pred))

      Precisión en datos de prueba: 0.8510670965760786
      Reporte de clasificación:
                    precision    recall  f1-score   support

                 0       0.85      0.97      0.91      4945
                 1       0.83      0.48      0.61      1568

          accuracy                           0.85      6513
         macro avg       0.84      0.72      0.76      6513
      weighted avg       0.85      0.85      0.84      6513
```

Fuente: Angel Peña (2025).

Finalmente, aplicamos el modelo entrenado a datos nunca vistos para hacer predicciones. Cargamos el archivo con los nuevos datos, limpiamos los nombres de columnas, convertimos las variables categóricas a numéricas y se predicen los ingresos para los nuevos registros.

Figura 9.

```
[11]: # Cargar el archivo con nuevos datos (sin variable objetivo)
ruta_nuevos = r"C:\Users\king1\tareasPython\Practica autonoma 4\Ingresos_nuevos_datos.xlsx"
X_new = pd.read_excel(ruta_nuevos)

# Limpiar nombres de columnas si es necesario
X_new.columns = X_new.columns.str.strip()

# Convertir variables categóricas a numéricas
for col in X_new.columns:
    if X_new[col].dtype == 'object':
        X_new[col] = le.fit_transform(X_new[col])

# Predecir ingresos para los nuevos registros
y_new_pred = grid_svc.predict(X_new)

print("Predicciones para nuevos datos:")
print(y_new_pred)

Predicciones para nuevos datos:
[0 0 0 0 0 0 1 1 0 1 0 0 0 1]
```

Fuente: Angel Peña (2025).

Conclusión

La implementación de este proyecto permitió construir un modelo predictivo con una precisión del 85.1%, demostrando la efectividad del algoritmo SVM para problemas de clasificación binaria. A lo largo de la actividad, se siguieron buenas prácticas en ciencia de datos, como la división estratificada de los datos y la optimización de hiperparámetros mediante validación cruzada. Si bien el modelo mostró un buen desempeño, se identificó que la clase minoritaria (>50K) presentó un recall más bajo, lo que sugiere oportunidades de mejora mediante técnicas de balanceo de datos o el uso de otros algoritmos. Este ejercicio sirvió como un ejemplo completo del flujo de trabajo en un proyecto de machine learning, desde el preprocesamiento hasta la generación de predicciones en nuevos datos. Los resultados obtenidos refuerzan la importancia de un enfoque metodológico en la implementación de modelos predictivos.